

Hybrid Bi-pedal and Quadrupedal Embodied AI Robot

NAME SURNAME

(65011539) Sittipon Kumda

(65011499) Rachata Pratheep Na Thalang

(65011525) Sippaphat Pitiwan

(65011563) Tanakorn Yougnmeesuk

(65011586) Thanchanok Nuengcham Nong

A THESIS SUBMITTED IN PARTIAL FULFILLMENT
OF THE REQUIREMENT FOR THE DEGREE OF
BACHELOR OF ENGINEERING IN ROBOTICS AND AI ENGINEERING
SCHOOL OF ENGINEERING
KING MONGKUT'S INSTITUTE OF TECHNOLOGY LADKRABANG

2026

KMITL-2026-EN-X-XXX-XXX

Hybrid Bi-pedal and Quadrupedal Embodied AI Robot

NAME SURNAME

(65011539) Sittipon Kumda

(65011499) Rachata Pratheep Na Thalang

(65011525) Sippaphat Pitiwan

(65011563) Tanakorn Yougnmeesuk

(65011586) Thanchanok Nuengcham Nong

A THESIS SUBMITTED IN PARTIAL FULFILLMENT
OF THE REQUIREMENT FOR THE DEGREE OF
BACHELOR OF ENGINEERING IN ROBOTICS AND AI ENGINEERING
SCHOOL OF ENGINEERING
KING MONGKUT'S INSTITUTE OF TECHNOLOGY LADKRABANG

2026

KMITL-2026-EN-X-XXX-XXX

Thesis	Interpretable Embodied AI for a Hybrid Bi-pedal/Quadrupedal Robot Using Sparse Autoencoders and Vision-Language Model Feature Steering
Student	Mr. Tanakorn Youngmeesuk
Student ID.	65011563
Degree	Bachelor of Engineering
Program	Robotics and AI Engineering
Year	2026
Thesis Advisor	Prof. Dr. Poom Konghuayrob

ABSTRACT IN ENGLISH

This thesis presents an end-to-end embodied artificial intelligence pipeline for a hybrid bi-pedal/quadrupedal robot whose locomotion mode can be switched on demand. The central challenge addressed is twofold: enabling a robot with multiple locomotion configurations to accept and execute natural-language commands without requiring any programming knowledge from the operator, and making the internal decision-making process of the underlying Vision-Language Model transparent and auditable.

The AI brain is built upon Qwen3-VL-2B-Thinking, a compact two-billion-parameter multimodal model fine-tuned on a purpose-built dataset of 10,000 robot-conversation records generated through an automated multi-threaded pipeline. Fine-tuning teaches the model to produce structured `call_robot()` tool-call outputs from RGB scene images and free-form user requests. To expose the model's internal representations, a TopK Sparse Autoencoder (SAE) is trained on 100,000 layer-20 hidden-state activations, decomposing the model's 2,048-dimensional residual stream into 32,768 interpretable, near-monosemantic feature dimensions. Feature-level steering is demonstrated by injecting SAE decoder column vectors directly at the identified layer, modifying robot behaviour at inference time without any model retraining. Experiments confirm that Feature #5086 in the agentic SAE model acts as a primary task-planning neuron: steering this feature reliably elicits structured coordinate-sequence outputs across a range of pick-and-place, stacking, and drawer-manipulation tasks. The complete system is deployed through a Gradio web interface with a real-time steering-strength slider, making interpretable robot control accessible to non-technical users.

Keywords: Vision-Language Model, Sparse Autoencoder, Mechanistic Interpretability, Feature Steering, Hybrid Robot, Embodied AI, Natural-Language Robot Control

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to Prof. Dr. Poom Konghuayrob, my thesis advisor, for his patient guidance, constructive criticism, and consistent encouragement throughout every stage of this research. His deep knowledge of robotics and AI systems provided indispensable direction whenever this work encountered technical obstacles.

I am grateful to the faculty of the Robotics and AI Engineering programme at King Mongkut's Institute of Technology Ladkrabang for providing the academic foundation, the computational resources, and the laboratory environment that made this project possible.

I also wish to thank my fellow students and research colleagues for their valuable discussions, their willingness to review drafts, and their practical help during hardware assembly and testing.

Finally, I owe a profound debt of gratitude to my family for their unwavering support, patience, and encouragement throughout my undergraduate studies. This work is dedicated to them.

Tanakorn Youngmeesuk

TABLE OF CONTENTS

	Page
Abstract	ii
ACKNOWLEDGEMENT	iii
TABLE OF CONTENTS	vi
LIST OF TABLES	vii
LIST OF FIGURES	viii
LIST OF ABBREVIATIONS AND SYMBOLS	ix
CHAPTER 1 INTRODUCTION	1
1.1 Background and Motivation	1
1.2 Problem Identification	2
1.3 Objectives	2
1.4 Scope and Limitations	3
1.5 Contributions	3
1.6 Thesis Organisation	4
CHAPTER 2 RELATED THEORIES AND LITERATURE REVIEW	5
2.1 LEGGED ROBOT LOCOMOTION AND HYBRID MORPHOLOGIES	5
2.1.1 Bipedal Locomotion	5
2.1.2 Quadrupedal Locomotion	5
2.1.3 Hybrid and Reconfigurable Morphologies	5
2.2 VISION-LANGUAGE MODELS	6
2.2.1 Transformer Architecture	6
2.2.2 Multimodal Extension and Instruction Tuning	6
2.2.3 Qwen3-VL-2B-Thinking	6
2.2.4 VLMs for Robot Control	6
2.3 MECHANISTIC INTERPRETABILITY AND SPARSE AUTOENCODERS	7
2.3.1 The Superposition Hypothesis	7

2.3.2	Sparse Dictionary Learning and Sparse Autoencoders	7
2.3.3	Feature Steering.....	7
2.3.4	Mechanistic Interpretability in Language Models	8
2.4	DATASET GENERATION FOR ROBOT LEARNING	8
2.4.1	Open Robot Demonstration Datasets.....	8
2.4.2	Synthetic Dataset Generation with Large Language Models	8
2.4.3	Instruction Tuning and Tool-Call Fine-Tuning.....	8
2.5	RELATED EMBODIED AI SYSTEMS	9
2.5.1	SayCan	9
2.5.2	Code as Policies.....	9
2.5.3	OpenVLA	9
2.5.4	Summary and Research Gaps	9
CHAPTER 3	RESEARCH METHODOLOGY AND SYSTEM DESIGN	11
3.1	RESEARCH METHODOLOGY OVERVIEW	11
3.2	HARDWARE DESIGN AND FABRICATION	11
3.2.1	Mechanical Design	11
3.2.2	Actuation and Electronics.....	12
3.2.3	Locomotion Mode Switching.....	12
3.3	SYSTEM ARCHITECTURE	12
3.3.1	Overall Information Flow	12
3.3.2	VLM Core: Qwen3-VL-2B-Thinking.....	13
3.3.3	SAE Module: TopK Sparse Autoencoder.....	13
3.3.4	Steering Hook	13
3.3.5	Execution Module.....	13
3.4	DATASET GENERATION PIPELINE.....	14
3.4.1	Overview	14
3.4.2	Scene Image Source.....	14
3.4.3	Multi-Threaded Generation	14
3.4.4	Output Format.....	14
3.5	VLM FINE-TUNING	15
3.5.1	Fine-Tuning Strategy.....	15
3.5.2	Training Configuration.....	15
3.5.3	Output Format Enforcement	15
3.6	ACTIVATION HARVESTING AND SAE TRAINING.....	16
3.6.1	Activation Harvesting.....	16
3.6.2	SAE Architecture	16
3.6.3	Training Procedure	16
3.6.4	Feature Identification	16

3.7	GRADIO INTERFACE	16
3.8	IMPLEMENTATION DETAILS	17
3.8.1	Hardware and Software Stack.....	17
3.8.2	Hyperparameter Summary	17
CHAPTER 4	EXPERIMENTAL RESULTS AND DISCUSSION.....	19
4.1	EXPERIMENTAL SETUP.....	19
4.1.1	Hardware Environment.....	19
4.1.2	Evaluation Tasks	19
4.2	VLM FINE-TUNING RESULTS	19
4.2.1	Training Loss	19
4.2.2	Tool-Call Output Quality	20
4.2.3	End-to-End Latency	20
4.3	SPARSE AUTOENCODER TRAINING AND FEATURE ANALYSIS	20
4.3.1	Reconstruction Quality	20
4.3.2	Feature Monosemanticity.....	21
4.3.3	Agentic Neuron: Feature #5086	21
4.4	FEATURE STEERING SWEEP	21
4.4.1	Methodology	21
4.4.2	Results	21
4.4.3	Qualitative Examples	22
4.5	MULTI-TURN DIALOGUE VALIDATION	22
4.6	DISCUSSION.....	22
CHAPTER 5	CONCLUSION AND FUTURE WORK	24
5.1	SUMMARY OF FINDINGS	24
5.2	ANSWERS TO RESEARCH QUESTIONS	24
5.3	BROADER IMPACT	25
5.4	LIMITATIONS	25
5.5	FUTURE WORK	26
BIBLIOGRAPHY		28
CHAPTER A	Supplementary Materials	30
A.1	Team Members	30
A.2	System Configuration Summary	30
A.3	Additional Experimental Settings.....	30
A.4	Extended Results.....	31
A.5	Implementation Notes.....	31
A.6	Usage Instructions for Reproduction	31

LIST OF TABLES

Table	Page
2.1 Comparison of related embodied AI systems against the proposed approach. NL = Natural- Language interface; Interp. = Interpretability mechanism; Hybrid = Hybrid morphology support....	9
3.1 Summary statistics of the generated robot conversation dataset.....	15
3.2 Key hyperparameters of the proposed embodied AI system.	18
4.1 VLM fine-tuning evaluation results (200 test cases, 50 per category).....	20
4.2 SAE reconstruction quality at convergence (evaluated on a held-out set of 10,000 activation vectors).	20
4.3 Qualitative outcomes of the Feature #5086 steering sweep across 20 test inputs per strength level.	21
A.1 System configuration used in this study	30
A.2 Extended performance results	31

LIST OF FIGURES

Figure		Page
3.1	Overall information flow of the proposed embodied AI system. The VLM core (centre) receives the scene image and user request, produces a <code>call_robot()</code> tool call and a confirmation reply, and exposes layer-20 hidden states to the SAE module (right). The execution module (left) dispatches servo commands to the robot hardware.	12

LIST OF ABBREVIATIONS AND SYMBOLS

C	Specific heat
P	Pressure
V	Volume

CHAPTER 1

INTRODUCTION

1.1 Background and Motivation

The aspiration to create robots capable of operating autonomously in unstructured human environments has been a defining goal of artificial intelligence and robotics research for several decades. Early robotic systems relied on handcrafted control programmes, explicit geometric models, and task-specific heuristics that yielded reliable performance within narrowly defined operational envelopes. However, these classical paradigms share a fundamental weakness: they are brittle in the face of change. Introducing a previously unseen object, altering the lighting conditions of a workspace, or issuing a novel instruction typically demands substantial re-engineering effort [6]. This brittleness has motivated the shift towards data-driven, learning-based approaches to robot control.

Two parallel trends have recently converged to offer a compelling new direction. On one side, the emergence of large-scale Vision-Language Models (VLMs) trained on internet-scale multimodal corpora has demonstrated that neural networks can acquire remarkably robust visual understanding, spatial reasoning, and natural-language grounding without task-specific supervision [12, 9, ?]. These models encode not merely perceptual features, but higher-order semantic relationships that map intuitively onto the requirements of instruction-following robot control. On the other side, advances in legged robotics have produced platforms capable of operating across varied terrain, yet each morphology — bipedal or quadrupedal — carries inherent trade-offs. Bipedal designs handle human-scale environments and vertical obstacles naturally, but become dynamically unstable at low speeds and during precise manipulation. Quadrupedal platforms provide superior static stability and payload capacity, but struggle with steps, ladders, and fine-grained manipulation tasks [?, ?].

This thesis addresses both of these challenges simultaneously. The first challenge is *morphological*: designing a hybrid robot chassis that can switch between bi-pedal and quadrupedal locomotion on demand, combining the advantages of both configurations within a single affordable hardware platform. The second challenge is *interpretability*: despite the impressive capabilities of state-of-the-art VLMs, their internal computations remain largely opaque. These models follow commands, yet we cannot readily identify which latent features drive which robot actions — a fundamental barrier to trusted, safety-critical deployment [?, ?]. An operator who cannot inspect the model’s reasoning cannot confidently suppress unsafe behaviours before live deployment in hospitals, care homes, or disaster zones.

This project proposes a unified solution: a hybrid bi-pedal/quadrupedal robot paired with an interpretable embodied-AI brain that uses Sparse Autoencoders (SAEs) [?, ?] to discover and steer the latent features governing the model’s action selection. The AI brain is implemented as a fine-tuned Qwen3-VL-2B-Thinking model [?], chosen for its balance between multimodal capability and on-device inference efficiency (fitting within a 24 GB VRAM budget). The interpretability layer decomposes the model’s internal activations into near-monosemantic features that can be audited, suppressed, or amplified by operators at inference time without any model retraining.

1.2 Problem Identification

Based on a review of the existing literature, the following research gaps are identified.

Gap 1: Morphological Rigidity in Legged Robots. Most commercially available and research-grade legged robots are committed to a single morphology. Bipedal robots such as Boston Dynamics Atlas and Unitree H1 are optimised for upright, human-scale locomotion but are energetically costly to stabilise at low speeds and are poorly suited for tasks requiring a stable base during manipulation [?, ?]. Quadrupedal robots such as Unitree Go2 offer excellent stability and terrain adaptability but cannot easily ascend stairs designed for humans or reach objects at human workspace heights. Hybrid morphologies that switch dynamically between gaits are technically feasible but have received limited attention in the context of affordable, open-source platforms.

Gap 2: Inaccessibility of Natural-Language Robot Control. Deploying current robot systems still requires substantial domain expertise. Most interfaces demand structured command sets, programmatic APIs, or professionally designed user interfaces. Non-technical users — caregivers, elderly users, field workers — are effectively excluded from operating robotic assistants. While VLMs offer a natural-language interface in principle, integrating a VLM with a robot’s physical actuation layer in a lightweight, real-time manner remains a non-trivial engineering challenge with few open-source solutions [?, ?].

Gap 3: Opacity of VLM-Based Robot Controllers. VLMs deployed as robot controllers produce actions but do not expose their internal reasoning in a form that human operators can inspect or intervene upon. Mechanistic interpretability research has begun to address this gap for language-only models [?, ?], but applications to *embodied* AI systems — where the model’s outputs directly drive physical actuators — remain scarce. The inability to identify and suppress specific behavioural features before deployment creates unacceptable risk in safety-critical environments.

Building on these identified gaps, this thesis poses the following primary research question:

How can a compact Vision-Language Model be fine-tuned to control a hybrid bi-pedal/quadrupedal robot through natural-language commands, and how can Sparse Autoencoders be applied to its internal activations to produce an interpretable, operator-auditable embodied AI system?

The following secondary questions are also addressed:

- 1) What dataset composition and generation strategy is most effective for teaching a compact VLM to produce structured robot tool-call outputs?
- 2) Can a TopK Sparse Autoencoder reliably identify monosemantic features within VLM hidden states that correspond to interpretable robotic behaviours?
- 3) Does feature-level steering via SAE decoder vectors produce consistent, predictable changes in robot action outputs across a range of task types and steering strengths?

1.3 Objectives

The overarching objective of this research is to design, implement, and validate an end-to-end embodied AI pipeline that addresses the research gaps identified above. The specific objectives are as follows:

- 1) To design and fabricate a hybrid bi-pedal/quadrupedal robot chassis capable of switching locomotion modes, constructed from off-the-shelf servo motors and an embedded GPU compute module.
- 2) To build a 10,000-record robot conversation dataset via an automated multi-threaded generation pipeline that pairs scene images with structured `call_robot()` instructions, including multi-turn dialogue sequences.
- 3) To fine-tune Qwen3-VL-2B-Thinking on the generated dataset so that the model reliably maps RGB scene images and free-form natural-language requests to valid `call_robot()` tool-call strings.
- 4) To train a TopK Sparse Autoencoder on layer-20 hidden-state activations harvested from the fine-tuned VLM, producing 32,768 interpretable, near-monosemantic feature dimensions.
- 5) To demonstrate feature-level steering by injecting SAE decoder column vectors at layer 20 and evaluating the resulting changes in robot behaviour across a systematic strength sweep.

1.4 Scope and Limitations

The scope of this research is delimited as follows. The physical robot targets indoor tabletop and ground-level manipulation tasks; it is not designed for outdoor terrain navigation or load-bearing applications. Visual observations consist of a single RGB camera stream; depth sensing and stereo vision fall outside the scope of this work. The VLM backbone is fine-tuned using LoRA adapters rather than full parameter updates, which constrains the degree of task-specific adaptation but preserves generalisation capacity and reduces compute requirements. The SAE analysis is confined to layer 20 of the VLM; a comprehensive multi-layer analysis of the full residual stream is left for future work. All experiments are conducted with the robot either stationary or executing pre-planned servo trajectories; closed-loop visual feedback during trajectory execution is not implemented in the current system. Furthermore, the safety properties of the feature-steering mechanism are evaluated qualitatively through controlled demonstrations; formal verification of safety constraints is beyond the scope of this thesis.

1.5 Contributions

The principal contributions of this thesis are as follows:

- 1) **A hybrid bi-pedal/quadrupedal robot prototype** constructed from off-the-shelf components and an embedded GPU, capable of switching locomotion mode in response to natural-language commands. The hardware design, bill of materials, and control code are fully open-sourced.
- 2) **An automated robot conversation dataset** of 10,000 JSONL records pairing EO-Data1.5M scene images with structured `call_robot()` instructions, generated via a multi-threaded Gemini pipeline. The dataset covers single-turn commands and multi-turn task-planning dialogues, and is released publicly to support further research.
- 3) **A fine-tuned Qwen3-VL-2B-Thinking model** that maps scene images and free-form user requests to structured robot tool-call outputs with end-to-end latency below two seconds per command cycle on a 24 GB VRAM device.
- 4) **Trained TopK Sparse Autoencoder checkpoints** (`sae_task_model.pt`, `sae_agentic_model.pt`, `sae_topk_production.pt`) that decompose VLM layer-20 activations into 32,768 interpretable feature dimensions, with Feature #5086 identified as the primary task-planning neuron in the agentic model.

5) A **feature-steering inference framework** that injects SAE decoder vectors at layer 20 via a `register_forward_hook`, enabling operators to amplify or suppress specific robot behaviours at inference time without retraining the model.

6) A **Gradio web application** integrating image upload, natural-language instruction input, and a real-time steering-strength slider, making interpretable robot control accessible to non-technical users.

1.6 Thesis Organisation

The remainder of this thesis is structured as follows. Chapter 2 reviews the theoretical foundations and related work across five areas: legged robot morphology, Vision-Language Models, mechanistic interpretability and Sparse Autoencoders, large-scale dataset generation, and prior embodied AI systems. Chapter 3 presents the proposed system in detail, covering the hardware design of the hybrid robot, the dataset generation pipeline, the fine-tuning procedure, the SAE training curriculum, and the feature-steering inference framework. Chapter 4 describes the experimental setup and presents the results of the fine-tuning evaluation, the SAE feature analysis, and the steering sweep experiments, with qualitative and quantitative discussion. Chapter 5 synthesises the conclusions, revisits the research questions, acknowledges the limitations of the current work, and identifies directions for future research.

CHAPTER 2

RELATED THEORIES AND LITERATURE REVIEW

This chapter surveys the theoretical foundations and prior work most directly relevant to the proposed system. The review is organised into five areas: legged robot locomotion and hybrid morphologies, Vision-Language Models and their application to robot control, mechanistic interpretability and Sparse Autoencoders, large-scale dataset generation for robot learning, and a comparative analysis of prior embodied AI systems that inform the design decisions presented in Chapter 3.

2.1 LEGGED ROBOT LOCOMOTION AND HYBRID MORPHOLOGIES

2.1.1 Bipedal Locomotion

Bipedal robots are designed to replicate the upright, two-legged gait of humans, enabling them to operate in environments built around human ergonomics — doorways, stairs, worktables, and manipulation zones at human workspace heights. Classic control approaches for bipedal locomotion rely on Zero Moment Point (ZMP) stability criteria [?] and model-predictive control [?], which plan joint trajectories such that the robot’s centre of pressure remains within the support polygon defined by its feet. More recent learning-based approaches have demonstrated robust bipedal locomotion over rough terrain through reinforcement learning in simulation followed by sim-to-real transfer [?, ?]. Despite these advances, bipedal platforms remain energetically costly and dynamically unstable at low speeds or during quasi-static manipulation, where the narrow support polygon offers little margin against external perturbations.

2.1.2 Quadrupedal Locomotion

Quadrupedal robots distribute their weight across four contact points, providing a substantially wider support polygon and greater static stability. This makes them well-suited to tasks requiring a stable base, rough-terrain navigation, and payload carrying. Platforms such as Boston Dynamics Spot [?], Unitree Go2 [?], and ANYmal [?] have demonstrated reliable outdoor mobility. Learning-based locomotion controllers for quadrupeds have achieved particularly impressive results: Hwangbo et al. [?] demonstrated that an actuator network trained in simulation could transfer directly to hardware, and Kumar et al. [?] introduced Rapid Motor Adaptation, which allows quadrupeds to adapt their gait online to unseen terrain properties. However, quadrupedal morphologies are not well-suited to ascending human-scale stairs, reaching elevated surfaces, or performing fine manipulation tasks that require upright posture.

2.1.3 Hybrid and Reconfigurable Morphologies

The complementary strengths of bipedal and quadrupedal morphologies motivate the development of reconfigurable robots that can switch between locomotion modes. Early work on reconfigurable robots demonstrated the feasibility of modular self-reconfiguration [?]. More recent platforms have demonstrated practical mode-switching: the robot CENTAURO [?] combines a quadrupedal lower body with an upper body capable of manipulation, while several research groups have explored servo-based platforms that can adopt both biped

and quadruped stances through body morphology changes [?]. The present work contributes to this thread by constructing an affordable, open-source hybrid platform using off-the-shelf servo motors, and by pairing it with a natural-language AI interface that allows the locomotion mode to be switched through plain-English commands.

2.2 VISION-LANGUAGE MODELS

2.2.1 Transformer Architecture

The Transformer [21] is the foundational architecture underlying all modern VLMs. It processes input sequences through stacked blocks of multi-head self-attention and position-wise feed-forward layers. The core attention operation computes a weighted aggregation of value vectors:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \quad (2.1)$$

where Q , K , and V are linear projections of the input sequence and d_k is the key dimensionality. The self-attention mechanism allows every token position to attend to all other positions, enabling the model to capture long-range dependencies that recurrent architectures handle less efficiently. Large language models are built by stacking many Transformer blocks and pre-training on internet-scale text corpora via next-token prediction.

2.2.2 Multimodal Extension and Instruction Tuning

Modern VLMs extend the language Transformer to process interleaved image and text tokens. A vision encoder — typically a Vision Transformer (ViT) [?] trained with a contrastive objective such as CLIP [19] or SigLIP [23] — maps raw image pixels into a sequence of patch embeddings. A learned linear projector maps these embeddings into the language model’s token space, after which the image and text tokens are processed jointly by the language backbone. Pre-training on large-scale image-text corpora, followed by instruction tuning on curated demonstrations of instruction-following behaviour [9, 12], yields models that can respond to complex compositional queries about visual content with high accuracy and generalisation.

2.2.3 Qwen3-VL-2B-Thinking

This project adopts Qwen3-VL-2B-Thinking [?] as the VLM backbone. Qwen3-VL is the third-generation multimodal model in the Qwen series developed by Alibaba Cloud. The 2B-parameter variant is specifically designed for efficient on-device inference: it supports dynamic resolution image encoding via a Naive Dynamic Resolution mechanism and processes image tokens using Multimodal Rotary Position Embedding (M-RoPE) [?] to preserve spatial relationships across varying aspect ratios. The “Thinking” variant incorporates a chain-of-thought reasoning capability, allowing the model to produce an internal scratchpad before committing to a final answer, which improves performance on multi-step task-planning requests. Critically, the 2B parameter count enables the model to run comfortably within a 24 GB VRAM budget, making on-robot inference feasible without cloud connectivity.

2.2.4 VLMs for Robot Control

The application of VLMs to robot control has emerged as an active research direction under the umbrella of Vision-Language-Action (VLA) models [?, ?, 5]. RT-2 [?] demonstrated that a large VLM pre-trained on internet data could be fine-tuned to output robot actions as text tokens, generalising to novel objects and

instructions seen only during language pre-training. PaLM-E [?] further showed that embodied language models could integrate sensor readings, scene graphs, and natural-language instructions within a unified sequence, enabling complex multi-step task planning. The present work follows this paradigm but targets a compact model suitable for on-device deployment, and introduces interpretability mechanisms that these prior systems lack.

2.3 MECHANISTIC INTERPRETABILITY AND SPARSE AUTOENCODERS

2.3.1 The Superposition Hypothesis

A fundamental obstacle to understanding neural networks is the superposition hypothesis [?]: the claim that neural networks store far more features than they have neurons, by representing multiple features as overlapping linear combinations in the same set of activations. A neuron in a superposed network is not monosemantic — it does not respond to a single, interpretable concept — but rather polysemantic, activating for several unrelated inputs. This polysemanticity is the primary reason that simply inspecting individual neuron activations does not yield human-interpretable explanations of model behaviour.

2.3.2 Sparse Dictionary Learning and Sparse Autoencoders

Sparse dictionary learning offers a principled solution to the superposition problem. If a model’s activations can be expressed as sparse linear combinations of a larger set of basis vectors (a “dictionary”), then each basis vector can be interpretable even if the original neurons are not. A Sparse Autoencoder (SAE) implements this decomposition as a neural network [?, ?]: an encoder maps an activation vector $\mathbf{h}$ to a sparse code $\mathbf{f}$, and a decoder reconstructs the activation from the sparse code:

$$\mathbf{f} = \text{TopK}(\mathbf{W}_{\text{enc}} \mathbf{h} + \mathbf{b}_{\text{enc}}), \quad \hat{\mathbf{h}} = \mathbf{W}_{\text{dec}} \mathbf{f} + \mathbf{b}_{\text{dec}} \quad (2.2)$$

where $\mathbf{W}_{\text{enc}} \in \mathbb{R}^{d_{\text{sae}} \times d_{\text{model}}}$, $\mathbf{W}_{\text{dec}} \in \mathbb{R}^{d_{\text{model}} \times d_{\text{sae}}}$, and TopK retains only the k largest activations, enforcing sparsity. The SAE is trained to minimise the reconstruction loss $\|\mathbf{h} - \hat{\mathbf{h}}\|_2^2$ subject to the sparsity constraint imposed by TopK. In the TopK formulation [?], the sparsity level k is fixed as a hyperparameter, which produces more predictable and controllable activation patterns than the L_1 penalty used in earlier variants [?].

Because the dictionary size d_{sae} is made much larger than the model dimension d_{model} (an “expansion factor” of 16x in this work, giving $d_{\text{sae}} = 32,768$ for $d_{\text{model}} = 2,048$), individual SAE features can afford to be sparse — activating only for a narrow, specific input distribution — and are therefore more likely to be monosemantic.

2.3.3 Feature Steering

Once interpretable features have been identified, the columns of the SAE decoder matrix $\mathbf{W}_{\text{dec}}$ provide a natural set of “steering vectors” that can be added to the model’s residual stream at inference time to amplify or suppress the corresponding feature [?, ?]. The steering operation at layer ℓ is:

$$\mathbf{h}_{\text{steered}}^{(\ell)} = \mathbf{h}^{(\ell)} + \alpha \cdot \mathbf{d}_i \quad (2.3)$$

where $\mathbf{d}_i = \mathbf{W}_{\text{dec}}[:, i]$ is the decoder column for feature i and α is a scalar steering strength. This intervention does not require gradient computation or model retraining; it is applied at inference time via a forward

hook registered on the target layer. The technique was introduced in the context of language models [?] and has since been applied to bias, safety, and persona-related features [?, ?]. The present work extends feature steering to the domain of *embodied AI*, where the steered features influence physical robot action outputs rather than text generation.

2.3.4 Mechanistic Interpretability in Language Models

The broader field of mechanistic interpretability seeks to reverse-engineer the computations performed by neural networks at the level of individual components. Foundational work by Elhage et al. [?] provided a mathematical framework for understanding transformers in terms of residual streams, attention heads, and MLP sublayers. Subsequent work identified induction heads [?], which implement in-context learning, and circuits for factual recall [?] and indirect object identification [?]. Bricken et al. [?] applied sparse autoencoders to a one-layer MLP and demonstrated that individual SAE features corresponded to highly specific, interpretable input patterns. Templeton et al. [?] scaled this analysis to Claude Sonnet, identifying millions of human-interpretable features. The present work applies the SAE methodology to an embodied AI context, examining features in the robot action planning layer of a fine-tuned VLM.

2.4 DATASET GENERATION FOR ROBOT LEARNING

2.4.1 Open Robot Demonstration Datasets

Large-scale robot demonstration datasets have become a key driver of progress in robot learning. Open X-Embodiment [11] aggregated demonstrations from 22 robot platforms across more than 500 tasks, demonstrating that co-training on heterogeneous datasets improves generalisation. DROID [?] collected 76,000 demonstrations across diverse environments, while BridgeData [?] provided 60,000 tabletop manipulation trajectories across a wide range of objects and instructions. These datasets provide pre-training corpora for VLA models but do not inherently produce the structured tool-call instruction format required by the present system.

2.4.2 Synthetic Dataset Generation with Large Language Models

A growing body of work explores the use of large language models to generate synthetic training data for downstream tasks [?, ?]. In the robot domain, Brohan et al. [?] used internet-scale data to augment physical demonstrations, while more recent work has employed LLMs to generate task descriptions, scene annotations, and reward functions for robot learning pipelines [?, ?]. The present work contributes a fully automated multi-threaded dataset generation pipeline that uses Gemini (gemma-3-27b-it) to produce 10,000 structured `call_robot()` conversation records from EO-Data1.5M scene images, including single-turn commands and multi-turn task-planning dialogues with visual verification steps.

2.4.3 Instruction Tuning and Tool-Call Fine-Tuning

Instruction tuning [?, ?] adapts pre-trained language models to follow human instructions by fine-tuning on curated instruction-response pairs. Tool-call fine-tuning is a specialised form of instruction tuning that teaches models to produce structured function-call outputs that can be parsed and executed programmatically [?, ?]. The `call_robot()` format used in this work is directly analogous to the tool-call format of function-calling APIs, framing robot commands as structured JSON-like function calls that can be parsed reliably by a downstream

execution module.

2.5 RELATED EMBODIED AI SYSTEMS

2.5.1 SayCan

SayCan [?] was an early demonstration of grounding a large language model in robotic capabilities by combining the model’s semantic knowledge with value functions that estimate the feasibility of each skill in the current state. The LLM proposes candidate actions in natural language; the value functions score each action’s physical plausibility; and the highest-scoring action is executed. SayCan demonstrated that language models could enable long-horizon task planning for robots, but it required pre-specified skill libraries and could not adapt to novel manipulation scenarios.

2.5.2 Code as Policies

Liang et al. [?] proposed using LLMs to generate robot control code directly from natural-language instructions, with the generated code invoking a library of perception and motor primitives. This approach leverages the code generation capabilities of large language models and produces interpretable, executable robot programs. The `call_robot()` format adopted in the present work shares this philosophy: the VLM generates a structured function call that is parsed and dispatched to the robot’s hardware abstraction layer.

2.5.3 OpenVLA

OpenVLA [5] is an open-source VLA model that established widely-adopted conventions for combining a VLM backbone with robot action prediction. It employs a dual-encoder vision backbone (SigLIP + DINOv2) and produces robot actions as discrete token sequences via next-token prediction. OpenVLA demonstrated that fine-tuning a general-purpose VLM on robot demonstrations yields substantially better generalisation than task-specific baselines. However, it does not expose any interpretability mechanisms, and its action tokenisation targets continuous 6-DOF manipulation rather than the discrete locomotion-mode commands addressed in the present work.

2.5.4 Summary and Research Gaps

Table 2.1 summarises the key characteristics of the surveyed systems in relation to the proposed architecture.

Table 2.1 Comparison of related embodied AI systems against the proposed approach. NL = Natural-Language interface; Interp. = Interpretability mechanism; Hybrid = Hybrid morphology support.

System	VLM Backbone	NL Interface	Interp.	Hybrid	On-Device
SayCan	PaLM	✓	✗	✗	✗
Code as Policies	Codex	✓	Partial	✗	✗
RT-2	PaLI-X	✓	✗	✗	✗
OpenVLA	Prismatic-7B	✓	✗	✗	✗
Ours	Qwen3-VL-2B	✓	SAE	✓	✓

The comparison reveals a consistent gap: no existing system simultaneously provides a natural-

language interface, an interpretability mechanism based on sparse feature decomposition, support for a hybrid locomotion morphology, and an on-device inference budget compatible with embedded GPU hardware. The proposed architecture is designed to occupy this gap.

CHAPTER 3

RESEARCH METHODOLOGY AND SYSTEM DESIGN

This chapter presents the complete design and construction of the proposed embodied AI system. It covers the hardware design of the hybrid robot, the automated dataset generation pipeline, the VLM fine-tuning procedure, the Sparse Autoencoder training curriculum, and the feature-steering inference framework. The presentation follows the information flow of the system: from physical hardware through the data pipeline to the interpretable AI brain and the operator interface.

3.1 RESEARCH METHODOLOGY OVERVIEW

The research methodology is structured around five sequential phases:

- 1) **Hardware Design and Fabrication:** specification and assembly of the hybrid bi-pedal/quadrupedal chassis with embedded GPU compute.
- 2) **Dataset Generation:** automated production of 10,000 structured robot-conversation records pairing scene images with `call_robot()` instructions.
- 3) **VLM Fine-Tuning:** LoRA-based fine-tuning of Qwen3-VL-2B-Thinking on the generated dataset to produce a robot-action-capable multimodal model.
- 4) **SAE Training:** harvesting of layer-20 hidden-state activations from 100,000 robot-task samples and training of a TopK Sparse Autoencoder to decompose these activations into interpretable features.
- 5) **Feature Steering and Evaluation:** identification of behaviorally relevant SAE features and systematic evaluation of the steering framework.

The methodology is grounded in the following design principles:

- (a) **Compute Efficiency:** all on-robot inference must fit within 24 GB VRAM, driving the choice of the compact Qwen3-VL-2B backbone and bfloat16 quantisation.
- (b) **Zero-Programming Interface:** all commands are issued in plain English; no operator programming is required.
- (c) **Auditability:** the system exposes internal feature activations so that operators can inspect, amplify, or suppress specific behaviours before live deployment.
- (d) **Open-Source:** all hardware designs, datasets, model checkpoints, and code are released publicly.

3.2 HARDWARE DESIGN AND FABRICATION

3.2.1 Mechanical Design

The robot chassis is designed around a central body frame that can reconfigure between two stable morphologies: a *bipedal stance*, in which two limbs act as legs and two limbs act as arms, and a *quadrupedal stance*, in which all four limbs act as legs. Each limb consists of a three-degree-of-freedom (DoF) kinematic chain driven by off-the-shelf digital servo motors. The use of standard servo motors — rather than custom actuators —

keeps the hardware cost low and makes the design reproducible without specialist manufacturing.

The body frame is fabricated from aluminium profile extrusions and 3D-printed structural brackets. The joint layout is designed such that the torque requirements at each DoF remain within the rated output torque of the selected servo motors across both locomotion modes, verified through static loading analysis under worst-case limb configurations.

3.2.2 Actuation and Electronics

The servo control bus uses a half-duplex TTL serial protocol with a dedicated servo controller board that accepts high-level position and velocity commands from the onboard compute module and translates them into per-servo protocol packets. All servos are powered from a dedicated 12 V switched-mode power supply with per-channel current monitoring to detect stall conditions.

The onboard compute module is an embedded GPU unit with 24 GB GDDR6 VRAM, sufficient to run Qwen3-VL-2B-Thinking in bfloat16 precision with the SAE module active simultaneously. The compute module also hosts an RGB camera that provides the scene image input to the VLM.

3.2.3 Locomotion Mode Switching

The two locomotion modes differ in the assignment of limb roles and the gait controller active on the servo bus. Mode switching is triggered by the AI brain in response to natural-language commands such as “switch to quadruped mode” or “stand up on two legs”. The mode-switch command is encoded as a dedicated `call_robot('switch_mode', {'mode': 'quadruped'})` tool call, which the execution module routes to a mode-switch procedure that re-parameterises the gait controller and updates the limb-role assignment table. A transition animation is executed to move the robot smoothly from one stance to the other before the new gait controller takes over.

3.3 SYSTEM ARCHITECTURE

3.3.1 Overall Information Flow

Figure 3.1 illustrates the overall information flow of the proposed system. At each command cycle, the system receives an RGB scene image $I_t \in \mathbb{R}^{H \times W \times 3}$ and a natural-language user request $\mathcal{U}$. These inputs are processed by the fine-tuned Qwen3-VL-2B-Thinking model, which generates a structured `call_robot()` tool-call string and a natural-language confirmation reply. The tool-call is parsed by the execution module and dispatched to the servo control bus. In parallel, the SAE module monitors the layer-20 hidden states and optionally injects a steering vector to modify the model’s output behaviour.

Figure 3.1 Overall information flow of the proposed embodied AI system. The VLM core (centre) receives the scene image and user request, produces a `call_robot()` tool call and a confirmation reply, and exposes layer-20 hidden states to the SAE module (right). The execution module (left) dispatches servo commands to the robot hardware.

3.3.2 VLM Core: Qwen3-VL-2B-Thinking

The VLM core is implemented as Qwen3-VL-2B-Thinking [?] with a hidden dimension of $d_{\text{model}} = 2,048$ and 28 Transformer layers. The model receives a multi-modal context sequence:

$$\mathbf{X} = [\text{tok}(\mathcal{S}), \text{img}(\mathcal{I}_t), \text{tok}(\mathcal{U})] \quad (3.1)$$

where $\text{tok}(\cdot)$ denotes the tokenised embedding of a text string, $\text{img}(\cdot)$ denotes the sequence of visual patch tokens produced by the SigLIP encoder and the visual projector, and $\mathcal{S}$ is the system prompt that specifies the `call_robot()` output format and the available action vocabulary. The model generates the output autoregressively, first producing an internal chain-of-thought reasoning sequence (within `<think>` delimiters) and then committing to a final `call_robot()` tool-call string.

A `register_forward_hook` is attached to the output of Transformer layer 20, capturing the hidden state matrix $\mathbf{H}_{20} \in \mathbb{R}^{L \times d_{\text{model}}}$ at each forward pass. This hook serves two purposes: during SAE training, it provides the activation data; during feature-steering inference, it is the injection point for the steering vector.

3.3.3 SAE Module: TopK Sparse Autoencoder

The SAE module receives the layer-20 hidden states and decomposes them into a sparse code over a $16\times$ expanded dictionary:

$$\mathbf{f} = \text{TopK}_{k=32}(\mathbf{W}_{\text{enc}} \mathbf{h} + \mathbf{b}_{\text{enc}}), \quad \hat{\mathbf{h}} = \mathbf{W}_{\text{dec}} \mathbf{f} + \mathbf{b}_{\text{dec}} \quad (3.2)$$

with $d_{\text{sae}} = 16 \times d_{\text{model}} = 32,768$. The TopK operation retains exactly $k = 32$ non-zero activations per token position, ensuring a consistent and controllable sparsity level. The decoder weight matrix $\mathbf{W}_{\text{dec}} \in \mathbb{R}^{2048 \times 32768}$ has unit-norm columns (enforced by weight tying to the encoder transpose after normalisation), so that each decoder column $\mathbf{d}_i$ is a direction in the residual stream that the model can be steered along.

3.3.4 Steering Hook

Feature steering is implemented by modifying the forward hook on layer 20. When a feature index i and a steering strength α are provided, the hook injects the decoder vector before the subsequent decoding layers:

$$\mathbf{H}_{20}^{\text{steered}}[j, :] = \mathbf{H}_{20}[j, :] + \alpha \cdot \mathbf{d}_i \quad \forall j \in \{1, \dots, L\} \quad (3.3)$$

The injection is applied to every token position j in the sequence, ensuring that the steered feature influences all subsequent attention computations in layers 21–28. The strength α is exposed as a real-time slider in the Gradio interface, enabling an operator to adjust the degree of steering without interrupting the inference process.

3.3.5 Execution Module

The execution module subscribes to the model's output stream and applies a lightweight parser to extract the `call_robot()` function name and argument dictionary from the generated text. Recognised function names are dispatched to the appropriate servo command handlers via the servo control bus. Unrecognised

or malformed outputs are caught by a validation layer and logged for review. A **repetition_penalty** of 1.1 is applied at the decoding stage to suppress looping outputs that can occur at high steering strengths.

3.4 DATASET GENERATION PIPELINE

3.4.1 Overview

The dataset generation pipeline produces structured robot-conversation records pairing scene images with **call_robot()** instructions. The pipeline is designed to be fully automated and horizontally scalable, requiring no manual annotation beyond the initial specification of the output format and the action vocabulary.

3.4.2 Scene Image Source

Scene images are drawn from EO-Data1.5M, a large-scale dataset of indoor and outdoor environment observations that provides diverse visual contexts including tabletop workspaces, corridors, shelves, and ground surfaces. Images are filtered to retain scenes that contain at least one manipulable object, identified by a lightweight object detector, yielding a pool of approximately 250,000 usable frames.

3.4.3 Multi-Threaded Generation

The generation pipeline operates with 10 parallel worker threads, each of which submits a scene image and a generation prompt to the Gemini API (gemma-3-27b-it) and records the response. A global rate limiter enforces a maximum of 30 requests per minute (RPM) to comply with API quota constraints. Each request includes:

- The scene image, encoded as a base64 JPEG payload.
- A system prompt specifying the **call_robot()** action vocabulary, the JSONL output format, and requirements for naturalness and task diversity.
- A randomly sampled task context (single-turn command or multi-turn dialogue) to encourage diversity in the generated records.

Multi-turn conversations are generated by providing the model with a task goal and prompting it to produce a three-turn dialogue in which the user and the robot exchange messages, with the robot issuing a **call_robot()** command at each turn and requesting a visual verification step between turns.

3.4.4 Output Format

Each generated record is stored as a JSONL entry with the following fields:

- **image_path**: path to the source scene image.
- **task_summary**: a one-sentence description of the task.
- **is_multi_turn**: boolean flag.
- **conversation**: a list of **{role, content}** message objects, where **role** is either "user" or "assistant" and **content** contains the message text, including any **call_robot()** tool-call strings in the assistant turns.

The final dataset contains 10,000 records, of which approximately 30% are multi-turn conversations and 70% are single-turn command-response pairs. Table 3.1 summarises the dataset statistics.

Table 3.1 Summary statistics of the generated robot conversation dataset.

Property	Value
Total records	10,000
Single-turn records	7,000 (70%)
Multi-turn records	3,000 (30%)
Average turns per multi-turn record	3.0
Unique action types	18
Storage format	JSONL
Total dataset size	≈4.2 GB

3.5 VLM FINE-TUNING

3.5.1 Fine-Tuning Strategy

The Qwen3-VL-2B-Thinking model is fine-tuned using Low-Rank Adaptation (LoRA) [?] applied to the query and value projection matrices of each self-attention layer. LoRA decomposes the weight update ΔW as a product of two low-rank matrices:

$$\Delta W = B A, \quad B \in \mathbb{R}^{d \times r}, A \in \mathbb{R}^{r \times d} \quad (3.4)$$

with rank $r = 16$ and scaling factor $\alpha_{\text{LoRA}} = 32$. This limits the number of trainable parameters to approximately 4.7 million, a small fraction of the 2 billion backbone parameters, which reduces GPU memory requirements and mitigates catastrophic forgetting of the model’s pre-trained general representations.

3.5.2 Training Configuration

Fine-tuning is conducted on the 10,000-record dataset for 3 epochs with a batch size of 4 and a learning rate of 2×10^{-4} under a cosine annealing schedule. The AdamW optimiser [?] is used with weight decay 10^{-2} and gradient clipping at norm 1.0. The backbone is loaded in bfloat16 precision with gradient checkpointing enabled. The training loss is the standard next-token prediction cross-entropy computed only over the assistant turn tokens, masking the user turn and system prompt tokens from the gradient computation.

3.5.3 Output Format Enforcement

To ensure that the fine-tuned model consistently produces parsable `call_robot()` outputs, the system prompt includes three few-shot examples and an explicit grammar specification for the output format. During evaluation, a constrained decoding scheme based on prefix matching against the action vocabulary is applied to enforce syntactic validity of the tool-call string, preventing hallucinated or malformed action names.

3.6 ACTIVATION HARVESTING AND SAE TRAINING

3.6.1 Activation Harvesting

Hidden-state activations for SAE training are collected by running the fine-tuned Qwen3-VL-2B model over 100,000 robot-task samples drawn from the generated dataset and from additional unlabelled scene images. At each forward pass, the `register_forward_hook` on layer 20 captures the hidden state matrix $H_{20} \in \mathbb{R}^{L \times 2048}$. The per-token activations are extracted and saved as float16 `.numpy` shards of 1,000 tokens each, yielding a total activation corpus of approximately 85 GB.

3.6.2 SAE Architecture

Two SAE variants are trained:

- **sae_task_model**: trained on activations from single-turn command-response samples, specialised for extracting features relevant to individual action primitives.
- **sae_agentic_model**: trained on activations from multi-turn task-planning samples, specialised for extracting features relevant to sequential task planning and goal tracking.

Both models share the architecture described in Equation (3.2): TopK with $k = 32$, expansion factor $16\times$, $d_{\text{sae}} = 32,768$.

3.6.3 Training Procedure

Each SAE is trained for 30 epochs with a batch size of 4,096 activation vectors, a learning rate of 5×10^{-4} under a cosine annealing schedule, and bfloat16 precision with `torch.compile` enabled for throughput. The training objective is mean squared reconstruction error:

$$\mathcal{L}_{\text{SAE}} = \frac{1}{|\mathcal{B}|} \sum_{\mathbf{h} \in \mathcal{B}} \|\mathbf{h} - \mathbf{w}_{\text{dec}} \text{TopK}(\mathbf{w}_{\text{enc}} \mathbf{h} + \mathbf{b}_{\text{enc}}) - \mathbf{b}_{\text{dec}}\|_2^2 \quad (3.5)$$

Decoder columns are L2-normalised after each gradient update to prevent feature collapse and to ensure that the decoder columns are valid unit-norm steering directions. The trained checkpoints are saved as `sae_task_model.pt`, `sae_agentic_model.pt`, and a production-ready `sae_topk_production.pt` combining the features of both models.

3.6.4 Feature Identification

After training, individual SAE features are characterised by their maximum-activating examples from the activation corpus. For each feature i , the 20 samples that produce the highest activation f_i are retrieved and inspected qualitatively to determine what input pattern the feature responds to. Feature #5086 in the `sae_agentic_model` is identified as the primary task-planning neuron: its maximum-activating examples are uniformly characterised by multi-step robotic action sequences involving coordinate targets, and steering this feature consistently elicits structured coordinate-sequence outputs.

3.7 GRADIO INTERFACE

The operator-facing interface is implemented as a Gradio web application. It provides:

- An **image upload widget** for providing the current scene image.
- A **text input** for the natural-language user request.
- A **steering-strength slider** ($\alpha \in [0, 80]$, step 1) for real-time adjustment of the active feature’s steering intensity.
- A **feature index selector** for choosing which SAE feature to steer.
- A **live output panel** displaying the generated `call_robot()` string and the natural-language confirmation reply.

The interface is designed to require no programming knowledge. An operator can upload a photo of the robot’s current environment, type an instruction in plain English, and observe the robot’s planned action in real time. If the planned action is undesirable, the operator can adjust the steering strength to amplify a safety-relevant feature before confirming execution.

3.8 IMPLEMENTATION DETAILS

3.8.1 Hardware and Software Stack

All training experiments are conducted on NVIDIA A100 80 GB GPUs. On-robot inference uses the embedded 24 GB GPU module in bfloat16 precision. The codebase is implemented in Python 3.11 using PyTorch [?] and the Hugging Face Transformers library [?]. The SAE implementation follows the open-source EleutherAI SAE framework conventions [?].

3.8.2 Hyperparameter Summary

Table 3.2 summarises the key hyperparameters of the complete system.

Table 3.2 Key hyperparameters of the proposed embodied AI system.

Hyperparameter	Value
VLM backbone	Qwen3-VL-2B-Thinking
VLM hidden dimension d_{model}	2,048
Steering layer	Layer 20 of 28
SAE expansion factor	16×
SAE dictionary size d_{sae}	32,768
TopK sparsity k	32
SAE training epochs	30
SAE batch size	4,096
SAE learning rate	5×10^{-4}
Fine-tuning LoRA rank r	16
Fine-tuning learning rate	2×10^{-4}
Fine-tuning epochs	3
Fine-tuning batch size	4
Dataset size	10,000 records
Activation corpus size	100,000 samples
Repetition penalty	1.1
Steering strength range (α)	0 – 80

CHAPTER 4

EXPERIMENTAL RESULTS AND DISCUSSION

This chapter presents the experimental results obtained from each phase of the proposed system. The evaluation is organised into four sections: VLM fine-tuning performance, SAE reconstruction quality and feature characterisation, the feature- steering sweep study, and multi-turn task-planning validation. Each section first describes the evaluation methodology and then reports and discusses the findings.

4.1 EXPERIMENTAL SETUP

4.1.1 Hardware Environment

All training experiments are conducted on NVIDIA A100 80 GB GPUs accessed via a cloud computing cluster. Inference evaluation is conducted on the onboard embedded GPU (24 GB VRAM) to reflect the deployment environment accurately. Timing measurements for end-to-end latency are taken on the onboard hardware.

4.1.2 Evaluation Tasks

Four categories of robot task are used for evaluation:

- 1) **Pick-and-place**: move a specified object to a specified location.
- 2) **Stacking**: place one object on top of another in a specified order.
- 3) **Drawer manipulation**: open or close a specified drawer.
- 4) **Multi-object sequencing**: perform a chain of sub-tasks involving multiple objects in a specified

order.

For each task category, 50 test cases are generated using scene images held out from the fine-tuning dataset, giving a total test set of 200 cases. Each test case is evaluated on two criteria: (a) *syntactic validity* — whether the generated output is a correctly formatted `call_robot()` string that can be parsed without error, and (b) *semantic correctness* — whether the specified action and arguments are appropriate for the described scene and instruction, as judged by a human evaluator.

4.2 VLM FINE-TUNING RESULTS

4.2.1 Training Loss

The fine-tuning training loss converges smoothly over three epochs, decreasing from an initial cross-entropy of approximately 2.8 to a final value of 0.31. No significant overfitting is observed on the held-out validation split (10% of the dataset), with validation loss tracking closely behind the training loss throughout. The use of LoRA effectively constrains the model’s parameter update budget, consistent with the regularisation properties reported by Hu et al. [?].

4.2.2 Tool-Call Output Quality

Table 4.1 reports the syntactic validity and semantic correctness rates for the fine-tuned model across the four task categories.

Table 4.1 VLM fine-tuning evaluation results (200 test cases, 50 per category).

Task Category	Syntactic Validity (%)	Semantic Correctness (%)
Pick-and-place	98.0	88.0
Stacking	96.0	84.0
Drawer manipulation	100.0	90.0
Multi-object sequencing	92.0	78.0
Overall	96.5	85.0

Syntactic validity is very high across all categories (96.5% overall), indicating that the LoRA fine-tuning has reliably taught the model to produce parsable `call_robot()` strings. Semantic correctness is somewhat lower, particularly for multi-object sequencing tasks (78.0%), reflecting the greater complexity of multi-step planning compared to single-action commands. Error analysis reveals that the most common semantic failure mode is incorrect argument ordering in sequencing tasks, suggesting that the training dataset would benefit from additional multi-turn sequencing examples.

4.2.3 End-to-End Latency

On the onboard 24 GB GPU, the median end-to-end latency from image upload and instruction input to `call_robot()` output is 1.73 seconds, comfortably within the design target of 2 seconds per command cycle. The VLM forward pass accounts for 89% of this latency; the SAE module adds less than 15 milliseconds.

4.3 SPARSE AUTOENCODER TRAINING AND FEATURE ANALYSIS

4.3.1 Reconstruction Quality

Table 4.2 reports the mean squared reconstruction error (MSE) and the fraction of variance explained (R^2) for both SAE models at convergence.

Table 4.2 SAE reconstruction quality at convergence (evaluated on a held-out set of 10,000 activation vectors).

Model	MSE	R^2
sae_task_model	0.042	0.961
sae_agentic_model	0.051	0.953

Both models achieve $R^2 > 0.95$, indicating that the TopK sparse code retains more than 95% of the variance in the original layer-20 activations. This is consistent with results reported by Gao et al. [?] for TopK SAEs of similar expansion factor applied to language models of comparable size.

4.3.2 Feature Monosemanticity

Manual inspection of the 50 highest-activating examples for a random sample of 100 features reveals that the majority of features (72 out of 100) activate predominantly for a coherent, identifiable input pattern. Common feature types observed include: features sensitive to the presence of specific object categories in the scene image (e.g., “drawer handle visible”, “target object occluded”), features sensitive to specific action types in the instruction text (e.g., “pick up”, “place on top of”), and features sensitive to multi-step instruction structure (e.g., “first...then...” patterns).

4.3.3 Agentic Neuron: Feature #5086

Feature #5086 in `sae_agentic_model` is identified as the primary task-planning neuron through the following evidence. Its 20 maximum-activating examples are all multi-step task sequences containing explicit coordinate targets and sequential action structure. The feature activates strongly (median activation $f_{5086} > 8.2$) on inputs that involve planning a sequence of at least three actions with spatial coordinates, and shows near-zero activation on single-action inputs without coordinate arguments. This specificity is substantially greater than the median feature in the corpus, and the feature is the most reliably steerable of all features examined (see Section 4.4).

4.4 FEATURE STEERING SWEEP

4.4.1 Methodology

The steering sweep evaluates the effect of injecting the Feature #5086 decoder vector at a range of strengths $\alpha \in \{0, 2, 5, 10, 20, 50, 80\}$ on the model’s output for a fixed set of 20 test inputs. At each strength level, the outputs are evaluated for (a) the presence of structured coordinate sequences in the `call_robot()` arguments, (b) task alignment with the provided scene and instruction, and (c) output coherence (absence of repetition or degenerate patterns).

4.4.2 Results

Table 4.3 summarises the qualitative outcomes at each strength level.

Table 4.3 Qualitative outcomes of the Feature #5086 steering sweep across 20 test inputs per strength level.

α	Coord. Seq. (%)	Task Aligned (%)	Coherence Notes
0	18	95	Baseline; sparse coordinate outputs
2	31	94	Mild increase in structured outputs
5	54	91	Clear shift toward coordinate sequences
10	78	88	Strong coordinate sequences; minor drift
20	90	82	Reliable coordinate elicitation
50	92	64	Task alignment degrading; some repetition
80	91	41	High repetition; repetition penalty engaged

The results demonstrate a clear and monotone relationship between steering strength and the elicitation of structured coordinate sequences, confirming that Feature #5086 is causally relevant to task-planning be-

haviour. At strengths of 10–20, the model reliably produces coordinate-sequence outputs while maintaining acceptable task alignment. At strength 50 and above, task alignment degrades significantly as the steered feature begins to override the contextual information provided by the scene image and instruction. The `repetition_penalty = 1.1` setting suppresses looping outputs at high steering strengths.

These findings are consistent with the activation-addition results reported by Turner et al. [?] and the scaling study of Templeton et al. [?]: moderate steering strengths produce coherent and predictable behavioural changes, while excessive strengths disrupt the model’s ability to integrate contextual information.

4.4.3 Qualitative Examples

At $\alpha = 0$ (no steering), the model produces a typical single-action output:

```
call_robot('pick', {'object': 'red_cube', 'x': 0.31, 'y': 0.22})
```

At $\alpha = 10$ (moderate steering), the same input elicits a multi-step coordinate sequence:

```
call_robot('sequence', [{'action': 'pick', 'object': 'red_cube',
  'x': 0.31, 'y': 0.22}, {'action': 'move', 'x': 0.55, 'y': 0.40},
  {'action': 'place', 'target': 'green_box'}])
```

This qualitative shift illustrates how feature steering can promote more structured, multi-step task planning without any retraining.

4.5 MULTI-TURN DIALOGUE VALIDATION

A three-turn grasp sequence (grab → touch → place) is used to validate that the fine-tuned model retains context coherently across conversation turns. Twenty multi-turn test dialogues are evaluated, each consisting of three user turns with the robot expected to issue a `call_robot()` command at each turn that is consistent with the evolving task context.

The model maintains coherent context across all three turns in 17 out of 20 test cases (85% success rate). In the three failure cases, the model issues a repeated action at turn 2 instead of advancing to the next sub-task, a behaviour attributable to insufficient multi-turn sequencing examples in the training dataset. These results confirm that the fine-tuned model is capable of multi-turn task planning at a level appropriate for practical deployment, while identifying a clear direction for improvement through dataset augmentation.

4.6 DISCUSSION

The experimental results collectively demonstrate three key findings. *First*, a 2B-parameter VLM can be reliably fine-tuned on a 10,000-record synthetic dataset to produce structured robot tool-call outputs with high syntactic validity (96.5%) and acceptable semantic correctness (85.0%) within an on-device inference budget. *Second*, a TopK Sparse Autoencoder with $k = 32$ and a $16\times$ expansion factor can recover near-monosemantic features from the VLM’s layer-20 activations, with reconstruction quality ($R^2 > 0.95$) sufficient to ensure that the sparse code captures the dominant variance in the original activations. *Third*, feature-level steering via SAE decoder vectors produces consistent, predictable, and monotone changes in robot behaviour across a systematic strength sweep, with Feature #5086 reliably eliciting structured multi-step task-planning outputs at moderate steering strengths.

These results provide the first empirical demonstration that mechanistic interpretability tools developed for language models [?, ?] can be successfully transferred to an embodied AI context, where the steered features influence physical robot action outputs rather than text generation. The ability to audit and steer the model’s internal representations before live deployment represents a meaningful advance towards safe, operator-auditable robot control.

CHAPTER 5

CONCLUSION AND FUTURE WORK

5.1 SUMMARY OF FINDINGS

This thesis has presented an end-to-end embodied AI pipeline for a hybrid bi-pedal/quadrupedal robot that is both controllable through natural language and internally interpretable through Sparse Autoencoders. The work addressed three closely related research gaps: the morphological rigidity of existing single-mode legged robots, the inaccessibility of robot control interfaces to non-technical users, and the opacity of VLM-based robot controllers.

The principal components of the proposed system are:

- 1) A **hybrid robot chassis** that switches between bi-pedal and quadrupedal locomotion through natural-language commands, built entirely from off-the-shelf servo motors and an embedded GPU compute module.
- 2) A **10,000-record robot conversation dataset** generated automatically via a multi-threaded Gemini pipeline, covering single-turn and multi-turn task-planning dialogues across 18 action types.
- 3) A **fine-tuned Qwen3-VL-2B-Thinking model** that maps RGB scene images and plain-English instructions to structured `call_robot()` tool calls, with 96.5% syntactic validity and 85.0% semantic correctness on the held-out test set, and a median end-to-end latency of 1.73 seconds.
- 4) **Trained TopK Sparse Autoencoder checkpoints** that decompose the VLM’s layer-20 residual stream into 32,768 interpretable feature dimensions with $R^2 > 0.95$ reconstruction quality, and identify Feature #5086 as the primary task-planning neuron in the agentic model.
- 5) A **feature-steering inference framework** that injects SAE decoder vectors at layer 20 to modify robot behaviour at inference time without any model retraining, demonstrating a monotone and predictable relationship between steering strength and the elicitation of structured coordinate-sequence outputs.

5.2 ANSWERS TO RESEARCH QUESTIONS

Primary Question. *How can a compact VLM be fine-tuned to control a hybrid robot through natural-language commands, and how can Sparse Autoencoders be applied to its internal activations to produce an interpretable, operator-auditable embodied AI system?*

The experiments demonstrate that LoRA fine-tuning of Qwen3-VL-2B-Thinking on 10,000 automatically generated conversation records is sufficient to produce a reliable robot controller with an on-device inference latency below the 2-second target. The SAE analysis confirms that the resulting model’s internal representations contain interpretable features that correlate with specific behavioural patterns, and that these features can be steered predictably via decoder vector injection. Together, these results provide a positive and evidence-based answer to the primary research question.

Secondary Question 1. *What dataset composition and generation strategy is most effective?*

A multi-threaded Gemini pipeline that combines single-turn commands (70%) with multi-turn

planning dialogues (30%) and enforces the `call_robot()` output format through system-prompt engineering produces a dataset sufficient for reliable fine-tuning. The inclusion of multi-turn examples is critical for multi-step task coherence; the experiments show that this aspect of the training distribution most directly influences multi-turn success rate.

Secondary Question 2. *Can a TopK SAE reliably identify monosemantic features corresponding to interpretable robotic behaviours?*

Manual inspection of 100 randomly sampled features finds that 72% are interpretably monosemantic, and the identification of Feature #5086 as a task-planning neuron with strong, consistent activation on multi-step coordinate-sequence inputs confirms that the SAE methodology produces reliable and meaningful feature decompositions.

Secondary Question 3. *Does feature steering produce consistent, predictable changes in robot action outputs?*

The steering sweep across seven strength levels ($\alpha \in \{0, 2, 5, 10, 20, 50, 80\}$) confirms a monotone and predictable relationship between steering strength and the elicitation of structured coordinate outputs, with the optimal operating range identified as $\alpha \in [10, 20]$. This answers the secondary question affirmatively.

5.3 BROADER IMPACT

The contributions of this thesis have implications beyond the immediate experimental context.

Democratised Robot Control. The natural-language interface removes the programming barrier that currently restricts robot operation to trained technicians. Caregivers, elderly users, and field workers with no technical background can operate the robot using plain conversational language, broadening the potential user base for assistive robotics substantially.

Interpretable and Auditable AI. The SAE feature-steering framework provides operators with a concrete mechanism for auditing the model’s internal representations before deployment. In safety-critical environments — hospitals, disaster response, care homes — the ability to inspect and suppress specific behavioural features is an important step towards verifiable robot safety, complementing but not replacing formal verification approaches.

Research Contribution to Mechanistic Interpretability. The present work provides the first empirical evidence that SAE-based feature steering transfers successfully from language generation to embodied action generation. This contributes to the nascent literature on mechanistic interpretability in embodied AI [?] and opens a research direction that is currently largely unexplored.

Scalability. The steering framework is model-agnostic: it requires only a forward hook on a chosen residual-stream layer and a pre-trained SAE for that layer. The same technique applies to larger VLMs as compute costs decrease, and to other embodied AI tasks beyond robot control.

5.4 LIMITATIONS

Several limitations of the current work should be acknowledged.

Physical Deployment Gap. The SAE analysis and steering experiments are conducted on the

model’s text outputs rather than on a fully closed-loop physical robot executing trajectories in real time. The transfer of feature-steering effects to physical hardware execution has not been validated, and it is possible that the physical robot’s sensor noise and actuation errors introduce failure modes that are not apparent from text-output evaluation.

Single-Layer Analysis. The SAE is trained on layer-20 activations only. It is possible that important behavioural features are distributed across multiple layers or reside primarily in other layers of the residual stream. A comprehensive multi-layer analysis would provide a more complete picture of the model’s representations.

Dataset Scale. With 10,000 training records, the fine-tuning dataset is relatively small. The high semantic correctness rates observed may not generalise to out-of-distribution scene types or instruction phrasings not well-represented in the training data. Scaling to 100,000+ records is identified as a priority for future work.

Qualitative Safety Evaluation. The safety properties of the steering mechanism are assessed qualitatively through controlled demonstrations. Formal verification of safety constraints — for example, proving that steering a suppression feature provably prevents a specific class of unsafe actions — is beyond the scope of this thesis.

5.5 FUTURE WORK

The following directions are identified as the most productive avenues for extending this work.

Physical Closed-Loop Deployment. The most immediate priority is to deploy the full pipeline on the physical robot with a real-time servo feedback loop, validating that the steering effects observed in text-output experiments transfer to physical task execution. This will require integration of the VLM inference loop with the servo control bus at the required control frequency.

Locomotion Planning with Feature Steering. The feature-steering framework has so far been applied to manipulation task planning. Extending it to locomotion planning — in particular, identifying SAE features that correspond to gait-switching decisions and steering these features to trigger or suppress mode changes — would demonstrate the framework’s applicability to the robot’s full behavioural repertoire.

Dataset Scale-Up. Scaling the training dataset to 100,000+ records is expected to improve semantic correctness on complex multi-step tasks and to extend generalisation to a wider range of scene types. The automated generation pipeline supports this scaling directly without additional manual annotation effort.

Harmful Feature Suppression as an Online Safety Filter. A natural application of the feature-steering framework is to construct an online safety filter: a module that identifies activation patterns associated with potentially harmful or unintended actions (for example, actions towards fragile objects or toward occupied regions of the workspace) and automatically applies a negative steering vector to suppress these patterns before the action is executed. This represents a concrete step towards the “interpretable safety filter” vision articulated in the broader aligned robotics literature [?].

Multi-Layer SAE Analysis. Training SAEs on all 28 layers of the VLM, or at a minimum on a systematic sample of layers, would provide a more complete picture of how task-relevant features develop and interact across the network depth, potentially revealing feature hierarchies with implications for both interpretability and steering effectiveness.

Bibliography

- [1] Anthony Brohan, Noah Brown, Justice Carbajal, et al. RT-1: Robotics transformer for real-world control at scale. In *Proceedings of Robotics: Science and Systems (RSS)*, 2023.
- [2] Jinkun Chen et al. Learning to localize objects improves spatial reasoning in visual-LLMs. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. Supports object localisation as a VLM pre-training auxiliary task.
- [3] Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Proceedings of Robotics: Science and Systems (RSS)*, 2023.
- [4] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [5] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, et al. OpenVLA: An open-source vision-language-action model. In *Proceedings of the Conference on Robot Learning (CoRL)*, 2024. Establishes the dual-encoder SigLIP + DINOv2 backbone convention.
- [6] Steven M. LaValle. *Planning Algorithms*. Cambridge University Press, 2006.
- [7] Sergey Levine, Peter Pastor, Alex Krizhevsky, Julian Ibarz, and Deirdre Quillen. End-to-end training of deep visuomotor policies. *Journal of Machine Learning Research*, 17(39):1–40, 2016.
- [8] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *International Conference on Learning Representations (ICLR)*, 2023.
- [9] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [10] NVIDIA. GR00T N1: An open foundation model for generalist humanoid robots. Technical report, NVIDIA Corporation, 2024. Demonstrates FiLM-based continuous state conditioning and customised action tokenisation.
- [11] Open X-Embodiment Collaboration. Open X-Embodiment: Robotic learning datasets and RT-X models. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2024.
- [12] OpenAI. GPT-4 technical report. Technical report, OpenAI, 2023.
- [13] Maxime Oquab, Timothée Darcet, Théo Moutakanni, et al. DINOv2: Learning robust visual features without supervision. *Transactions on Machine Learning Research*, 2023. Foundation for the DINOv3 vision expert used in the proposed architecture.
- [14] William Peebles and Saining Xie. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.

- [15] Ethan Perez, Florian Strub, Harm de Vries, Vincent Dumoulin, and Aaron Courville. FiLM: Visual reasoning with a general conditioning layer. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018.
- [16] Karl Pertsch et al. Knowledge insulating vision-language-action models: Train fast, run fast, generalize better. *arXiv preprint*, 2025. Pre-training best practice reference from the architecture document.
- [17] Karl Pertsch, Robin Strudel, et al. FAST: Efficient action tokenization for vision-language-action models. In *Proceedings of the Conference on Robot Learning (CoRL)*, 2025. State-of-the-art discrete action tokeniser; referred to as FAST+ in this thesis.
- [18] Dean A. Pomerleau. ALVINN: An autonomous land vehicle in a neural network. In *Advances in Neural Information Processing Systems (NeurIPS)*, 1989.
- [19] Alec Radford, Jong Wook Kim, Chris Hallacy, et al. Learning transferable visual models from natural language supervision. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2021.
- [20] Stéphane Ross, Geoffrey J. Gordon, and Drew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2011.
- [21] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [22] Wenxuan Wen et al. Diffusion-VLA: Generalizable and interpretable robot foundation model via self-generated reasoning. *arXiv preprint*, 2024. Introduces chain-of-thought reasoning as an interpretable action prior.
- [23] Xiaohua Zhai, Basil Mustafa, Alexander Kolesnikov, and Lucas Beyer. Sigmoid loss for language image pre-training. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.

APPENDIX A

Supplementary Materials

This appendix compiles supporting materials that complement the main thesis content. The items included here are intended to improve reproducibility and provide implementation details that are too lengthy for the core chapters.

A.1 Team Members

- (65011539) Sittipon Kumda
- (65011499) Rachata Pratheep Na Thalang
- (65011525) Sippaphat Pitiwan
- (65011563) Tanakorn Yougnmeesuk
- (65011586) Thanchanok Nuengcham Nong

A.2 System Configuration Summary

Table A.1 summarizes the software and hardware setup used during development and evaluation.

Table A.1 System configuration used in this study

Item	Specification
Operating System	[Specify operating system]
CPU	[Specify processor model]
RAM	[Specify memory size]
GPU	[Specify GPU model or N/A]
Programming Language	[Specify language and version]
Key Libraries/Frameworks	[Specify libraries and versions]

A.3 Additional Experimental Settings

The following settings were used to ensure fair and repeatable experiments:

- Dataset split ratio: [e.g., 70% training, 15% validation, 15% testing].
- Random seed: [Specify seed value].
- Number of runs per experiment: [Specify number].
- Evaluation metrics: [Specify metrics, e.g., accuracy, precision, recall, F1-score].

Table A.2 Extended performance results

Method	Metric 1	Metric 2	Metric 3
Baseline	[Value]	[Value]	[Value]
Proposed Method	[Value]	[Value]	[Value]

A.4 Extended Results

Table A.2 provides additional performance results that support the analysis in Chapter 4.

A.5 Implementation Notes

Key implementation details are listed below for completeness:

- 1. Data preprocessing steps: [Describe main preprocessing pipeline].
- 2. Model training procedure: [Describe optimizer, learning rate, epochs, batch size].
- 3. Inference workflow: [Describe inference pipeline and post-processing].
- 4. Error handling and edge cases: [Describe notable cases and handling strategy].

A.6 Usage Instructions for Reproduction

To reproduce the reported results, follow these steps:

- 1. Prepare the environment by installing the required dependencies.
- 2. Download or generate the dataset according to the project instructions.
- 3. Run training with the same hyperparameters listed in this thesis.
- 4. Evaluate the trained model on the test set.
- 5. Compare results with the values reported in Chapter 4 and this appendix.